BLOOD SIMPLE

BIO-512 - Project

Due date: May 30th, 2024

Alice Granboulan,
Nicolas Moyne,
Noé Prat

Abstract

Blood Simple aim to help people understand their blood results.

Blood results are hard to interpret, especially for non-scientific people. With Blood Simple, they only have to upload a document or an image and an interactive window gives them well-documented insights on their blood results. This tool is all the more important as people become increasingly concerned with their own health, and incomprehensible test results can reveal to be a source of anxiety. Moreover, it can help patients understand their doctor's decisions and help them to improve their health. Lastly, it can help doctors to save some of their precious time by answering some of their patients' questions that do not represent a vital threat.

Blood Simple is be a user-friendly Web App, using image text recognition to extract blood test data from a document or an image. The bibliography supporting scientific knowledge is accessible to users. Descriptions of the various parameters and associated diseases are available for those wishing to learn more. The privacy of users is preserved as no data extracted from pdf is stored on a central server. The Web App will be constantly updated, taking into account users' questions and suggestions.

Table of contents

1 Introduction

A blood test is a laboratory analysis of a blood sample used to evaluate the functioning of various organs such as the liver, kidneys, thyroid, and heart, as well as to detect infections and genetic disorders. Additionally, it provides an overall assessment of an individual's health.

Once the sample is analyzed, the results are compiled into a blood test report. This report details the different components present in the blood and their levels. For those without a medical background, these reports can be complex and challenging to understand.

Understanding blood test results is crucial for people because it enables them to gain insights into their health status, identify potential health issues early, and make informed decisions about their medical care. Awareness and comprehension of these results can lead to better health management and proactive measures to maintain or improve one's well-being.

In addition, understanding blood test results can help doctors save valuable time by addressing patients' non-critical questions. When patients have a basic comprehension of their test results, they can interpret common findings on their own, reducing the need for consultations on minor concerns. This allows doctors to focus more on diagnosing and treating significant health issues, ultimately improving the efficiency and effectiveness of healthcare delivery. Additionally, it empowers patients to be more engaged in their health care, fostering a collaborative doctor-patient relationship.

2 Methods

As current LLMs do not provide reliable sources when answering a question, we decided to **build our own databases about blood parameters and related diseases**. We focused on two basic blood test parameters : Comprehensive Metabolic Panel (CMP) and Complete Blood Count (CBC), which gave us a total of 29 parameters. For each of them, we looked for the diseases related to up-regulation of the parameter and those related to its down-regulation. It gave a total of 77 diseases. For each disease listed, a simplified sourced description and sourced information on its epidemiology were found. All this information was stored in Excel files that were then used to generated Streamlit pages in a systematic manner thanks to a single Python script.

We built a first version of the webapp using **Flask** (after having considered Django, which was too complex for the project's time limit) for maximum flexibility. Flask is a lightweight and flexible web framework for Python that is designed for building Web Apps and APIs. It is highly customizable, but this flexibility was too great, Flask has a minimalistic core and we had to define the entire graphic charter.

Blood Simple had to be attractive and intuitive to use, so we switched to **Streamlit**. Streamlit is an open-source Python framework designed for creating and sharing custom Web Apps. It requires no user interface experience and enables reactive programming, two considerable assets for us given the short time available to build a prototype. Moreover, it works seamlessly with many popular Python libraries used in data science and machine learning, like NumPy,

Pandas, Scikit-learn, and TensorFlow.

To help users better understand their results, we first had to extract the data from their CMP or CBC. This extraction process involves two steps. The first step is **raw text extraction**. We provide users with an input button for their document that accepts .pdf, .jpg, .jpeg and .png formats. If the document is a .pdf with text fields, we use **native Python modules** to extract the text. For other file formats, such as .png, .jpg, and .jpeg, and for .pdf documents without text fields, where no native Python module can extract text directly from an image, we perform **Optical Character Recognition (OCR)** using the **PyTesseract framework**. Once this initial step is complete, we format the extracted data into a .json file using OpenAI tools, which allow us to get a response from ChatGPT that adheres to a predefined .json structure.

One of the key features of Blood Simple is a **chatbot** that has knowledge of both the user's blood results and the interpretation of CBC/CMP results. To achieve this, we cannot rely solely on **OpenAI API**, as ChatGPT does not have in-depth, well-sourced biological knowledge. To enhance ChatGPT's capabilities, we built a custom prompt using the .json files containing the user's results and the parameters/diseases interpretation. OpenAI provides tools to support instruction prompts like this, which enabled us to enhance the user experience by giving them access to a chatbot with more comprehensive knowledge than the standard ChatGPT.

3 Prototype

The final prototype takes as input a pdf or image with the results of a blood test (CMP or CB)C, allows the user to correct the extracted data, and displays a table with the abnormal values being highlighted as well as lists of up-regulated and down-regulated blood components. The user can click on each parameter to get more information about it. A parameter page contains a brief description of the parameter's function in the human body, and lists of potential diseases when values are above the reference range (up-regulation) or below (down-regulation). Information on each disease and its prevalence can be found on the "disease" pages.

All information is sourced with clickable links.

If the user prefers not to click on multiple pages to get all these insights, he can use the chatbot to get a summary of his abnormal values and potentially related diseases.

4 Learnings

One of our key learnings is the relative simplicity involved in building a large language model (LLM) chatbot. In an era where derivatives of ChatGPT are immensely popular, constructing our own chatbot has provided us with a deeper understanding of the distinctions between a mere ChatGPT clone and a newly trained LLM. Additionally, this process has offered us valuable insights into the methods of developing prototypes for this type of product, knowledge that will undoubtedly be beneficial in future endeavors. The small cost of ChatGPT API was also a great surprise and a good thing to learn.

We also learned a lot about creating a web page, switching between back-end, front-end and user experience in order to maximize efficiency and ergonomics, as well as reading documentation and researching similar features available on the Internet. We already knew that there were many Python packages available to facilitate web app development, but what we learned was that there is a spectrum from packages that are very flexible but more complex to handle, such as Django or even Flask, to packages that are more intuitive but less flexible, such as Streamlit. Therefore, we had to choose the solution best suited to the project's deadlines and resources.

Another teaching was how to synthesize complex scientific and medical concepts and explain them as clearly as possible, as well as to find an appropriate database design to store all the relevant information.

Prototyping has been a significant learning experience in this project. We discovered that using less flexible but more manageable frameworks can greatly enhance the development of a compelling prototype, not least by improving overall design and ease of use. This insight highlighted the advantages of choosing frameworks that align more closely with the specific needs of the project. The ease of use offered by such frameworks enabled us to concentrate more on refining the prototype's design and improving its user-friendliness, ultimately resulting in a neater, easier-to-use final product. The lessons learned from this experience will undoubtedly benefit our future projects. Understanding the trade-offs between flexibility and manageability, and recognizing the importance of selecting the right tools for the task at hand, has provided us with valuable insights. These will help us in to make more informed decisions in future endeavors, ensuring that we can effectively develop prototypes that are both functional and user-centered. This experience underlines the importance of tool selection during the prototyping phase, shaping our approach to future projects and contributing to more successful outcomes.